```
class MarketProjectionHandler ... {

 def serialize(event: Market.Event): Array[Byte] = {
    val proto = event match {                                  ❶
      case Market.Closed(marketId, result, _) =>
        projection.proto.MarketClosed(marketId, result)
      case Market.Opened(marketId, _, _) =>
        projection.proto.MarketOpened(marketId)
      case Market.Cancelled(marketId, reason) =>
        projection.proto.MarketCancelled(marketId, reason)
      case x =>
        log.info(s"ignoring event $x in projection")
        Empty.defaultInstance    }
    PbAny.pack(proto, "market-projection").toByteArray         ❷
 }
}
```

❶ Filters events and creates the proto

❷ Transforms the proto to a byte array

With all the parts that make up the process and serialization, you can create the handler, as shown in the next listing.

Listing 15.6 Market Kafka projection

```
class MarketProjectionHandler(
    system: ActorSystem[_],
    topic: String,
    producer: SendProducer[String, Array[Byte]])
    extends Handler[EventEnvelope[Market.Event]] {                 ❶

  val log = LoggerFactory.getLogger(classOf[MarketProjectionHandler])
  implicit val ec = ExecutionContext.global                        ❷

  override def process(
      envelope: EventEnvelope[Market.Event]): Future[Done] = ...   ❸

  def serialize(event: Market.Event): Array[Byte] = ...
  ...
}
```

❶ Extends from Handler, not JdbcHandler as in BetProjection

❷ This executor uses a pool with as many threads as the number of available processors.

❸ Serializes each event and pushes it to Kafka